

Autonomous APT Agent

林郁晁 112550126

許士宏 112550094

EXPLOIT STRATEGY

Vulnerability & Security Bypass



Vulnerability: Stack-based Buffer Overflow

- The blogic program processes /shared/config.data without boundary checks, allowing the exploit module to corrupt the stack frame and hijack the saved return address.



Bypassing NX / DEP (Phase II)

- Utilized Ret2Win (ROP) technique. Instead of injecting shellcode into the non-executable stack, control flow is redirected to the pre-existing execute_task function within the executable .text segment.



Bypassing ASLR (Phase III)

- Exploited the non-PIE (Position Independent Executable) configuration. While ASLR randomizes the stack layout, the main program's address space remains static.
- The Exploit Module uses pwntools ELF parsing to dynamically resolve the absolute addresses of execute_task and the ret gadget.

PAYLOAD ANATOMY

Structure of the Poisoned config.data



```
Prefix: user_input=  
    Required to satisfy the business logic parser.  
Command: /backdoor; #  
    The payload executes the backdoor binary. The # character acts as a shell comment to prevent the  
    trailing padding bytes from causing a syntax error in system().  
Padding: \x41 ('A') * N  
    Dynamically calculated size: (Offset - length of Command). Fills the buffer exactly up to the saved  
    return address.  
Alignment Fix (Optional): ret gadget address  
    Injected based on the Agent's retry strategy to shift RSP by 8 bytes, satisfying the x86-64 16-byte  
    stack alignment ABI requirement.  
Target Address: execute_task address  
    Overwrites the return address to trigger the exploit.
```

EC Workflow

- 1 Discovery
Round 1 starts with discovery(cyclic 256) instead of fixed exploitation
- 2 Coredump
Crash data is collected for triage analysis
- 3 Triage Recovery
Analyzes coredump and writes `/tmp/offset.txt` only on evidence
- 4 Exploit Retry
Next exploit round prioritizes the recovered triage offset

EXPLOIT.PY

Triage-First Decision Logic



Step 1: Check Offset

- Check whether `/tmp/offset.txt` exists
- Read and validate the recovered triage offset value



Step 2: Triage-Guided Path

- If a valid offset exists, immediately use triage-guided strategy
- Write payload using the recovered offset and return



Step 3: Fallback Discovery

- If no triage offset is available, send discovery payload
- Use recovered offset with two variants: non-aligned and ret-aligned

```
if triage offset is not None:  
    write payload  
    return  
if no triage offset:  
    send discovery payload
```

TRIAGE.PY

Evidence-Based Offset Recovery

Select Coredump

Selects the latest non-empty coredump from `/shared/coredump`

Read Crash Context

Reads the value at RSP-8 from the coredump, where the overwritten return address is likely located (Because ret may have advanced RSP by 8 bytes at crash time)

Search for Evidence

Searches for cyclic pattern evidence within the coredump

Compute or Abort

If evidence found: computes return-address offset and writes `/tmp/offset.txt`. If not found: no file written.

```
RSP = crash stack pointer
*(RSP-8) = overwritten return data
cyclic_offset = recovered pattern location
ret_offset = cyclic_offset + saved frame adjustment
```

DESIGN DECISION

Why Alignment Retry Is Needed

The recovered offset controls *where* the return address is overwritten — but **x86-64 stack alignment** may still affect function-call stability independently.

Recovered Offset

Triage pinpoints the exact return address location in the stack frame.

Variant 1: No Align

Direct jump to target (no alignment)

Variant 2: Ret-Align

Use ret gadget to fix stack alignment

Success

One of the two variants succeeds — ensuring robustness across different runtime stack states.

DEMO

Successful Run

1

Round 1

Exploit sends a **cyclic discovery payload**. Triage detects cyclic evidence near RSP.

2

Triage Analysis

Cyclic offset = **96** → corresponds to saved RBP.
Return address sits 8 bytes above: **ret_offset = 96 + 8 = 104**.
Writes `/tmp/offset.txt`.

3

Round 2

Exploit uses triage offset **104** with `align=False` (no ret gadget).

4

Round 3

Exploit retries triage offset **104** with `align=True` (ret gadget prefix) → **Success**.

```
Round 1: DISCOVERY
Triage: cyclic offset=96 -> ret_offset=104
Round 2: TRIAGE offset=104, align=False
Round 3: TRIAGE offset=104, align=True
Result: success
```

Thank You